# 03_two_layer_net_repo

March 24, 2026

# 1 Fully-Connected Neural Nets on CIFAR-10

This notebook adapts the original CS231n two-layer network assignment into a more repo-friendly walkthrough.

## 1.1 Goals

- implement affine and ReLU layers with separate forward and backward passes
- verify gradients numerically
- use convenience layer compositions such as affine-ReLU
- implement and test a two-layer fully-connected network
- train the model with the `Solver` abstraction
- inspect optimization curves and first-layer weights
- tune hyperparameters and reason about overfitting

## 1.2 Related source files

- `cs231n/layers.py`
- `cs231n/layer_utils.py`
- `cs231n/classifiers/fc_net.py`
- `cs231n/solver.py`
- `cs231n/data_utils.py`
- `cs231n/gradient_check.py`

## 1.3 0. What is the model in this notebook?

The main model is a **two-layer fully-connected network**:

$$X \longrightarrow XW_1 + b_1 \longrightarrow \mathrm{ReLU}(\cdot) \longrightarrow RW_2 + b_2 \longrightarrow \text{softmax loss.}$$

So although people call it a "two-layer network," what they usually mean is **two learnable affine layers**: 1. the hidden affine layer, 2. the output affine layer.

The ReLU in the middle is the nonlinearity that makes the model more expressive than a purely linear classifier.

## 1.4 1. Setup

The small path adjustment below makes the notebook easier to run locally from either the assignment root or a nearby notebook folder.

[ ]:
```
from __future__ import print_function

import sys
import time
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt

# Make sure the repository root containing `cs231n/` is importable.
cwd = Path().resolve()
candidates = [cwd] + list(cwd.parents[:3])
for cand in candidates:
    if (cand / "cs231n").exists():
        if str(cand) not in sys.path:
            sys.path.insert(0, str(cand))
        break

from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient,␣
↪eval_numerical_gradient_array
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0)
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

# Optional convenience for local development
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """Return the maximum relative error."""
    return np.max(np.abs(x - y) / np.maximum(1e-8, np.abs(x) + np.abs(y)))
```

### 1.5 2. Load the preprocessed CIFAR-10 data

We use the helper from the assignment, which already returns train / validation / test splits in the shapes expected by the fully-connected network code.

[3]:
```
# Load the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k, v in list(data.items()):
  print(('%s: ' % k, v.shape))
```

```
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

### 1.6 3. Affine layer: forward

Open `cs231n/layers.py` and implement `affine_forward`.

Recall that the affine layer reshapes each input example into one row and computes

$$\text{out} = X_{\text{flat}} W + b.$$

[4]:
```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),␣
 ↪output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference:  9.769849468192957e-10
```

### 1.7 4. Affine layer: backward

Now implement `affine_backward` and compare against numerical gradients.

[ ]:
```
# Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
```

```
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,␣
↪dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,␣
↪dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,␣
↪dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error:  5.399100368651805e-11
dw error:  9.904211865398145e-11
db error:  2.4122867568119087e-11
```

### 1.8 5. ReLU activation: forward

Implement the forward pass for the ReLU activation

$$\mathrm{ReLU}(x) = \max(0, x).$$

[5]:
```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,        ],
                        [ 0.,          0.,          0.04545455,  0.13636364,],
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,       ]])

# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference:  4.999999798022158e-08
```

### 1.9 6. ReLU activation: backward

Now implement `relu_backward` and verify it numerically.

[6]:
```
np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error:  3.2756349136310288e-12
```

### 1.9.1 Discussion: which activation functions can cause weak gradient flow?

A common issue in deep learning is that gradients can become zero or extremely small during backpropagation.

- **Sigmoid:** yes.

$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \sigma'(x) = \sigma(x)(1-\sigma(x)).$$

  When $x \gg 0$ or $x \ll 0$, the sigmoid saturates, so $\sigma'(x) \approx 0$.

- **ReLU:** yes, on part of its domain.

$$\mathrm{ReLU}(x) = \max(0, x), \qquad \mathrm{ReLU}'(x) = \begin{cases} 1, & x > 0, \\ 0, & x \le 0. \end{cases}$$

  So for negative inputs, the gradient is exactly zero.

- **Leaky ReLU:** much less so.

$$\mathrm{LeakyReLU}(x) = \begin{cases} x, & x > 0, \\ \alpha x, & x \le 0, \end{cases} \qquad \mathrm{LeakyReLU}'(x) = \begin{cases} 1, & x > 0, \\ \alpha, & x \le 0, \end{cases}$$

  where $\alpha > 0$ is small. The gradient on the negative side is small but still nonzero.

So in this list, **sigmoid** and **ReLU** can both create poor gradient flow, but for different reasons: sigmoid because of saturation at both tails, and ReLU because it shuts gradients off completely on the negative side.

## 1.10 7. Convenience blocks: affine followed by ReLU

Certain layer patterns appear so often that it is convenient to package them together. In `cs231n/layer_utils.py`, `affine_relu_forward` and `affine_relu_backward` combine those two steps into one reusable block.

[8]:
```
from cs231n.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w,␣
↪b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w,␣
↪b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w,␣
↪b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

### 1.11 8. Softmax loss layer

Implement `softmax_loss` in `cs231n/layers.py`. This should look very similar to the softmax work from the earlier linear-classifier notebook.

[9]:
```
np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)


dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,␣
↪verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should␣
↪be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing softmax_loss:
loss:  2.3025458445007376
dx error:  8.234144091578429e-09
```

### 1.12 9. Implement and test TwoLayerNet

Now complete the TwoLayerNet class in cs231n/classifiers/fc_net.py and run the checks below.

[10]:
```
np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
  [[11.53165108,  12.2917344,   13.05181771,  13.81190102,  14.57198434, 15.
 ↪33206765,  16.09215096],
   [12.05769098,  12.74614105,  13.43459113,  14.1230412,   14.81149128, 15.
 ↪49994135,  16.18839143],
   [12.58373087,  13.20054771,  13.81736455,  14.43418138,  15.05099822, 15.
 ↪66781506,  16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
```

```
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
  print('Running numeric gradient check with reg = ', reg)
  model.reg = reg
  loss, grads = model.loss(X, y)

  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
    print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))
```

```
Testing initialization …
Testing test-time forward pass …
Testing training loss (no regularization)
Running numeric gradient check with reg =  0.0
W1 relative error: 1.83e-08
W2 relative error: 3.20e-10
b1 relative error: 9.83e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg =  0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 9.09e-10
```

### 1.13 10. Train a baseline model with `Solver`

Read through `cs231n/solver.py` and use it to train a baseline two-layer net. A validation accuracy around 36% is a reasonable first target.

[11]:
```
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes)
solver = None

################################################################################
# TODO: Use a Solver instance to train a TwoLayerNet that achieves about 36% #
# accuracy on the validation set.                                              #
################################################################################
```

```python
solver = Solver(
    model,
    data,
    update_rule='sgd',
    optim_config={
        'learning_rate': 1e-3,
    },
    lr_decay=0.95,
    num_epochs=10,
    batch_size=200,
    print_every=100,
    verbose=True
)
solver.train()
################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

```
(Iteration 1 / 2450) loss: 2.301725
(Epoch 0 / 10) train acc: 0.187000; val_acc: 0.181000
(Iteration 101 / 2450) loss: 1.840451
(Iteration 201 / 2450) loss: 1.744794
(Epoch 1 / 10) train acc: 0.383000; val_acc: 0.418000
(Iteration 301 / 2450) loss: 1.529679
(Iteration 401 / 2450) loss: 1.543941
(Epoch 2 / 10) train acc: 0.444000; val_acc: 0.439000
(Iteration 501 / 2450) loss: 1.552313
(Iteration 601 / 2450) loss: 1.511572
(Iteration 701 / 2450) loss: 1.462240
(Epoch 3 / 10) train acc: 0.476000; val_acc: 0.444000
(Iteration 801 / 2450) loss: 1.515084
(Iteration 901 / 2450) loss: 1.462425
(Epoch 4 / 10) train acc: 0.495000; val_acc: 0.469000
(Iteration 1001 / 2450) loss: 1.376394
(Iteration 1101 / 2450) loss: 1.612014
(Iteration 1201 / 2450) loss: 1.612220
(Epoch 5 / 10) train acc: 0.509000; val_acc: 0.477000
(Iteration 1301 / 2450) loss: 1.427094
(Iteration 1401 / 2450) loss: 1.422479
(Epoch 6 / 10) train acc: 0.535000; val_acc: 0.480000
(Iteration 1501 / 2450) loss: 1.395185
(Iteration 1601 / 2450) loss: 1.405752
(Iteration 1701 / 2450) loss: 1.210371
(Epoch 7 / 10) train acc: 0.543000; val_acc: 0.508000
(Iteration 1801 / 2450) loss: 1.204663
(Iteration 1901 / 2450) loss: 1.364992
(Epoch 8 / 10) train acc: 0.524000; val_acc: 0.488000
(Iteration 2001 / 2450) loss: 1.294265
```

```
(Iteration 2101 / 2450) loss: 1.311599
(Iteration 2201 / 2450) loss: 1.323174
(Epoch 9 / 10) train acc: 0.575000; val_acc: 0.509000
(Iteration 2301 / 2450) loss: 1.209796
(Iteration 2401 / 2450) loss: 1.320206
(Epoch 10 / 10) train acc: 0.528000; val_acc: 0.504000
```

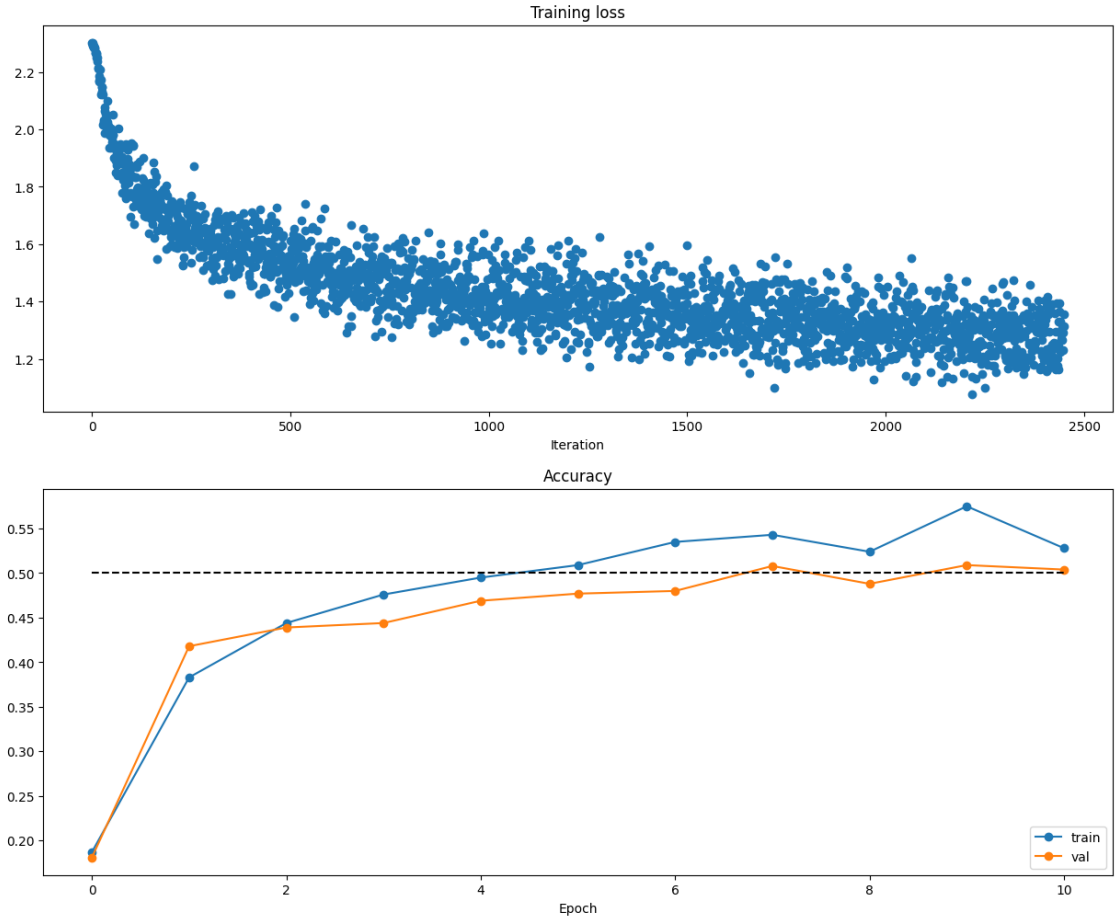
### 1.14 11. Inspect the training dynamics

A loss curve and train/validation accuracy curves often reveal whether the optimization is too slow, unstable, or beginning to overfit.

[12]:
```
# Run this cell to visualize training loss and train / val accuracy

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```




### 1.14.1 Visualize the first-layer weights

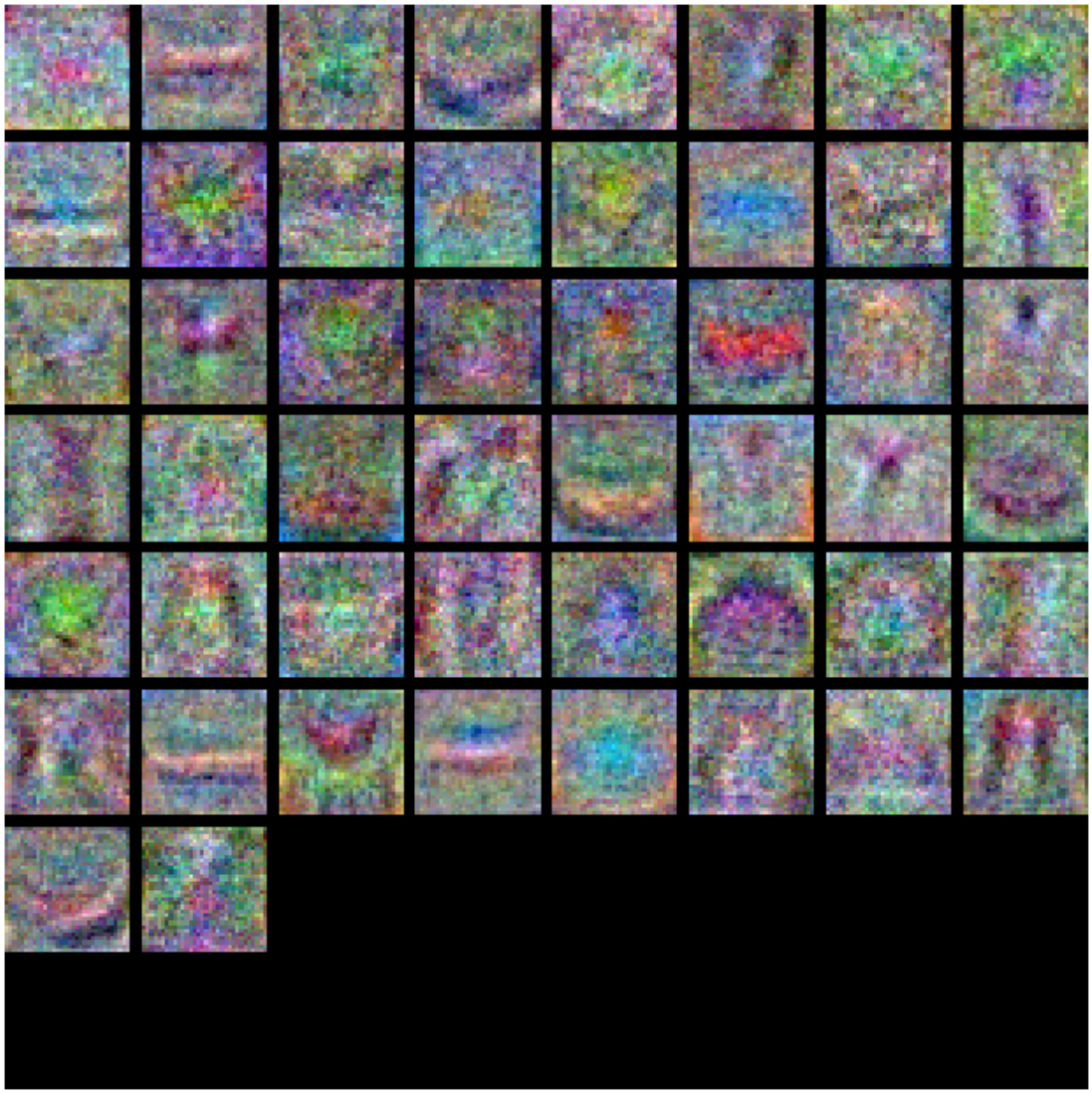
For image models, the first-layer weights often reveal whether the network has learned any meaningful low-level structure.

[13]:
```
from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



### 1.15 12. Hyperparameter tuning

Typical knobs here include hidden size, learning rate, regularization strength, number of epochs, and learning-rate decay.

A useful diagnostic is the gap between training and validation accuracy: - a **tiny gap** together with weak accuracy may indicate underfitting, - a **large gap** usually indicates overfitting.

[14]:
```
best_model = None
best_val_acc = -1
best_stats = None

################################################################################
```

```
# TODO: Tune hyperparameters using the validation set. Store your best trained ␣
↪#
# model in best_model.                                                          ␣
↪#
################################################################################

hidden_sizes = [100, 200]
learning_rates = [1e-3, 3e-3]
regularization_strengths = [0.25, 0.5]

for hs in hidden_sizes:
    for lr in learning_rates:
        for reg in regularization_strengths:

            model = TwoLayerNet(input_size, hs, num_classes, reg=reg)

            solver = Solver(
                model,
                data,
                update_rule='sgd',
                optim_config={'learning_rate': lr},
                lr_decay=0.95,
                num_epochs=10,
                batch_size=200,
                print_every=100,
                verbose=False
            )

            solver.train()

            val_acc = solver.best_val_acc
            train_acc = solver.train_acc_history[-1]

            print(
                f"hs={hs}, lr={lr}, reg={reg}, "
                f"train_acc={train_acc:.4f}, val_acc={val_acc:.4f}"
            )

            if val_acc > best_val_acc:
                best_val_acc = val_acc
                best_model = model
                best_stats = {
                    'hidden_size': hs,
                    'learning_rate': lr,
                    'reg': reg,
                    'train_acc': train_acc,
                    'val_acc': val_acc,
```

```
                }

print("best_val_acc:", best_val_acc)
print("best_stats:", best_stats)

################################################################################
#                                END OF YOUR CODE                              #
################################################################################
```

```
hs=100, lr=0.001, reg=0.25, train_acc=0.5800, val_acc=0.5160
hs=100, lr=0.001, reg=0.5, train_acc=0.5620, val_acc=0.5160
hs=100, lr=0.003, reg=0.25, train_acc=0.5480, val_acc=0.4910
hs=100, lr=0.003, reg=0.5, train_acc=0.4560, val_acc=0.4960
hs=200, lr=0.001, reg=0.25, train_acc=0.6090, val_acc=0.5350
hs=200, lr=0.001, reg=0.5, train_acc=0.5990, val_acc=0.5350
hs=200, lr=0.003, reg=0.25, train_acc=0.5790, val_acc=0.5030
hs=200, lr=0.003, reg=0.5, train_acc=0.5340, val_acc=0.5130
best_val_acc: 0.535
best_stats: {'hidden_size': 200, 'learning_rate': 0.001, 'reg': 0.25,
'train_acc': np.float64(0.609), 'val_acc': np.float64(0.535)}
```

## 1.16 13. Final evaluation

Evaluate the best model on the validation and test sets.

[15]:
```
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
```

```
Validation set accuracy:  0.535
```

[16]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Test set accuracy:  0.535
```

[17]:
```
# Save best model
best_model.save("best_two_layer_net.npy")
```

```
best_two_layer_net.npy saved.
```

## 1.17 14. Conceptual checklist

### 1.17.1 How can we reduce a large train-test accuracy gap?

If test accuracy is much lower than training accuracy, the model is likely **overfitting**.

The helpful moves among the listed choices are:

1. **Train on a larger dataset** — yes.
   More data makes it harder for the model to memorize quirks of the training set and usually improves generalization.

2. **Add more hidden units** — no, not as a cure for this gap.
   This increases model capacity, which often makes overfitting worse unless other controls are added.

3. **Increase the regularization strength** — yes.
   Stronger regularization discourages overly large weights and can reduce overfitting.

So the correct selections are **1 and 3**.